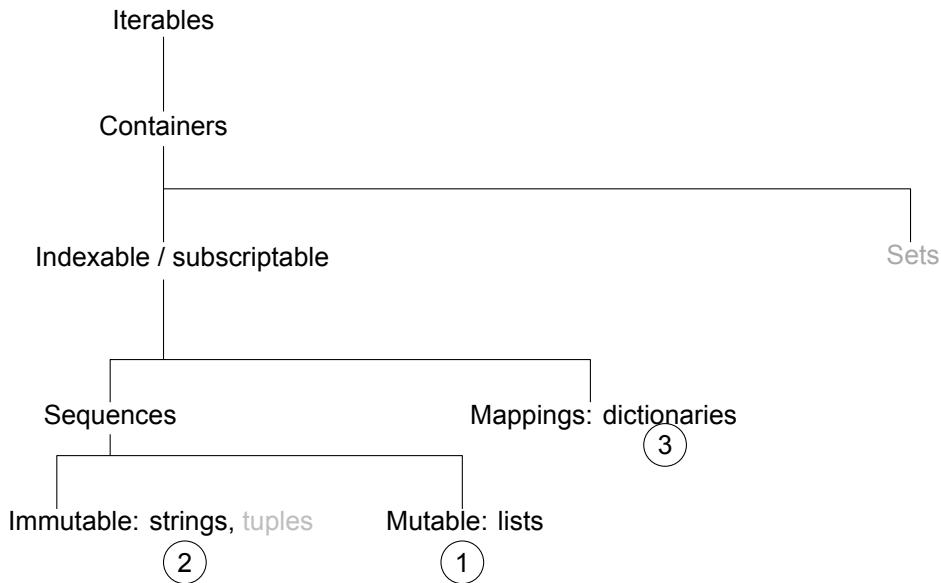


Python: container types (lists, strings, dictionaries)

Intro. to Computing

<https://mandarm.github.io/computing.html>

Container types



Container types

■ Lists: **mutable** sequence

- elements accessed using *indices*, and **can be changed**
- creating lists

```
l = [ 11, "y", 7.4 ]  
l = []           # empty list  
l = list(range(0,100,10)) # generally, list(iterable)  
numbers = [10, 16, -2, 22, -98]
```

LATER: using list comprehension

■ Strings (str) : **immutable** sequence

- elements accessed using *indices*, but **cannot be changed**

■ Dictionaries { 1:11, 2:"y", 3:7.4, 3.14:"pi" }

- elements accessed using *keys* (must be distinct)

■ Tuples (11, "y", 7.4)

■ Sets { 11, "y", 7.4 } — *not "subscriptable" (indexable)*

Lists: operations

(Applicable to immutable sequences also)

Operation	Result
<code>x in s</code>	True if an item of <code>s</code> is equal to <code>x</code> , else False ^{***}
<code>x not in s</code>	False if an item of <code>s</code> is equal to <code>x</code> , else True ^{***}
<code>s + t</code>	concatenation of <code>s</code> and <code>t</code> , does not change <code>s</code> , <code>t</code>
<code>s * n</code> or <code>n * s</code>	equivalent to concatenating <code>s</code> <code>n</code> times ^{††}
<code>len(s)</code>	length of <code>s</code>

^{***} For strings, `x` can also be a substring, e.g.,
`"gg" in "eggs" "ana" in "banana"`

^{††} **LATER:** may lead to unexpected behaviour

Exercise: Try using a negative value for `n`.

Lists: indexing and slicing

Indexing

negative indexes

s =

-n	-(n+1)	...	-2	-1
[10,	20,	...	40,	50,]
0	1	...	n-2	n-1

positive indexes

NOTE:
 $-0 \equiv 0$

Slicing: `s[start:end:step]`

- start: starting index, **default** 0, included in slice
- end: ending index, **default** `len(s)`, excluded from slice
- step: skip step elements
(negative step $\Rightarrow$ reverse direction, `step == 0` not permitted)
- Replace out of range values by 0 or `len(s)`, as appropriate.
- If `start >= end`, the slice is empty.

Note the difference with `start`, `stop` for `range()`.

Lists: more operations

(Applicable to mutable sequences only)

Operation	Result
<code>s[i] = x</code>	item <code>i</code> of <code>s</code> is replaced by <code>x</code>
<code>del s[i]</code>	removes item <code>i</code> of <code>s</code>
<code>s[i:j] = t</code>	slice of <code>s</code> from <code>i</code> to <code>j</code> is replaced by the contents of the <code>t</code>
<code>del s[i:j:k]</code>	removes the elements of <code>s[i:j:k]</code> from the list, <code>:k</code> optional same as <code>s[i:j:k] = []</code>
<code>s[i:j:k] = t</code>	elements of <code>s[i:j:k]</code> are replaced by those of <code>t</code> <code>:k</code> optional, but if not 1, <code>s[i:j:k]</code> and <code>t</code> must have same length
<code>s += t</code>	extends <code>s</code> with contents of <code>t</code>
<code>s *= n</code>	updates <code>s</code> with its contents repeated <code>n</code> times

Lists: some useful functions I

(Applicable to immutable sequences also)

- `s.count(val)`: number of occurrences of `val` in sequence `s`

- `s.index(val)`

 - `s.index(value, start)`

 - `s.index(value, start, stop)`

index of the **first** occurrence of `value` in sequence `s`, but **ERROR** if it does not occur

LATER: exceptions (errors) and how to handle them

Usage:

```
s = [ 0, 1, 2, 3, 2, 0, 1, 0, 3, 2]
s.count(0)
[ 0, 1, 2, 3, 2, 0, 1, 0, 3, 2].count(3)
```

Lists: some useful functions II

(Applicable to mutable sequences only)

- `s.append(value)`
- `s.pop(index)`: index optional, **default** is -1
get the item at index and remove it from the sequence.
- `s.remove(value)`: remove the first item that matches value, but **ERROR** if it does not occur
- `s.extend(sequence)`
- `s.insert(index, value)`
- `s.reverse()`, `reversed(s)`: **NOTE the difference!**

Lists: more useful functions

If the elements can be compared

- `min(l)` smallest item of `l`
- `max(s)` largest item of `s`
- `sort()`, `sorted()`
- `sum()` for numeric types

```
numbers = [10, 16, -2, 22, -98]
sorted(numbers)      # [-98, -2, 10, 16, 22]
reversed(numbers)    # <list_reverseiterator object at ...>
numbers.sort()       # changes IN PLACE
numbers.reverse()    # changes IN PLACE
```

Note the difference between `sorted()` (returns a list) and `reversed()` (returns an iterable).

Strings (`str` type)

Creating strings

- Single, double, triple quotes
- `str()` function applied to any variable or expression
(creates printable string representation of the variable / expression)
- Two adjacent string literals separated by only whitespace automatically joined into single string literal

```
s1 = 'allows embedded "double" quotes'
s2 = "allows embedded 'single' quotes"
s3 = '''Three single
(allows multiple embedded newlines and ") quotes'''
s4 = """Three double
(allows multiple embedded newlines and ') quotes"""
s5 = str([1, 2, 3, 4])
s6 = str({"sky" : "blue", "grass" : "green", "blood" : "red"})
s7 = "hello" ' world'
```

Digression: escape sequences, 'raw' strings

```
>>> s4
"Three double \n(allows multiple embedded newlines and ') quotes"
>>> print(s4)
>>> s = 'hi\n'
>>> print(s)
>>> len(s)
```

Escape sequences: character sequences starting with a backslash (\) that denote some other character that may be hard to type / represent directly

Examples:

- `\t` $\equiv$ tab (somewhat useful for lining things up in columns)
- `\r` $\equiv$ carriage return (takes cursor back to beginning of line)
(try `print("hello\nworld")` and `print("hellooooo\rworld")`)

Raw strings

- strings prefixed with `r`, e.g., `r"hi\n"`
- escape sequences treated literally, do **NOT** have special meaning
(try `print(r"hello\nworld")` and `print(r"hellooooo\rworld")`)

Lists: operations

(Applicable to immutable sequences also)

Operation	Result
<code>x in s</code>	True if an item of <code>s</code> is equal to <code>x</code> , else False ^{***}
<code>x not in s</code>	False if an item of <code>s</code> is equal to <code>x</code> , else True ^{***}
<code>s + t</code>	concatenation of <code>s</code> and <code>t</code> , does not change <code>s</code> , <code>t</code>
<code>s * n</code> or <code>n * s</code>	equivalent to concatenating <code>s</code> <code>n</code> times ^{††}
<code>len(s)</code>	length of <code>s</code>

^{***} For strings, `x` can also be a substring, e.g.,
`"gg" in "eggs"` `"ana" in "banana"`

^{††} **LATER:** may lead to unexpected behaviour

Exercise: Try using a negative value for `n`.

String functions: character types

Digression: non-English languages and Unicode

- `s.isalpha()`: all characters in `s` are letters,
Examples: `'Letters and spaces'.isalpha()`,
`'LettersOnly'.isalpha()`
- `s.islower()`: all characters in `s` are lowercase
- `s.isupper()`: as above
- `s.isdigit()`, `s.isnumeric()`
- `s.isalnum()`
- `s.isspace()`: blanks, tabs, newlines, etc.

`s` must have at least one character of the desired type

String functions: changing case

Strings are **immutable**; all functions return a modified copy.

- `s.upper()`: all cased characters are converted to uppercase
Question: Is `s.upper().isupper()` guaranteed to return `True`?
- `s.lower()`: as above
- `s.swapcase()`: uppercase characters are converted to lowercase and vice versa
- `s.capitalize()`: first character is capitalised, rest is lowercased

String functions: searching

end, or both start and end may be omitted

- `in` operator, e.g., `"ana" in "banana"`
- `s.find(substring, start, end)`: returns lowest index in the string where substring `sub` is found, -1 if `sub` is not found

Question: What is the difference between `find` and `index`?

- `s.rfind(substring, start, end)`: as above, but returns highest index
- `s.startswith(prefix, start, end)`
 - possible to check whether `s` with any of a number of prefixes by using a *tuple* of strings in place of `prefix`
- `s.endswith(suffix, start, end)`: as above
- `s.count(sub, start, end)`: returns number of *non-overlapping* occurrences of `sub` in `s` (or appropriate slice of `s`)

Question: What happens when `sub` is the empty string (`""`)?

String functions: replacing

- `s.removeprefix(somestring)`: if the string starts with `somestring`, remove it from the returned copy, otherwise return the original string `s`
- `s.removesuffix(somestring)`: as above
- `s.strip()` or `s.strip("some string")`: any leading / trailing characters that are in the specified string is removed
(if no arguments are provided, leading and trailing *whitespace* is removed)
Examples: " abcd ".strip(), " abcd ".strip("dcba"),
"ad abcd bc".strip("dcba")
- `s.lstrip()`, `s.rstrip()`: as above, but only leading (`lstrip`) OR trailing (`rstrip`) characters are removed
Question: What is the difference between `<lstrip(), removeprefix()>` and `<rstrip(), removesuffix()>`?
- `s.replace(oldstring, newstring, count)`: occurrences of `oldstring` are replaced by `newstring`
 - if `count` (optional) is not specified or negative, all occurrences are replaced; otherwise only the first `count` occurrences are replaced